Roles, Processes & Principles



DRIM

Software Engineering Roles

Software Engineering Roles

- Software Engineer
- Operations Engineer
- Security Engineer

DRIM

Software Engineer

- Requirements analysis
- System architecture and design
- Features implementation
- Bug fixing
- Code maintenance
- Auto-testing
- Performance optimization
- R&D

DRIM

Operations Engineer

- System architecture and design
- Deployment and release
- System monitoring and incident response
- Performance optimization
- Capacity planning and scalability
- Automation and tooling
- Disaster recovery and business continuity

DevOps

Software Engineer

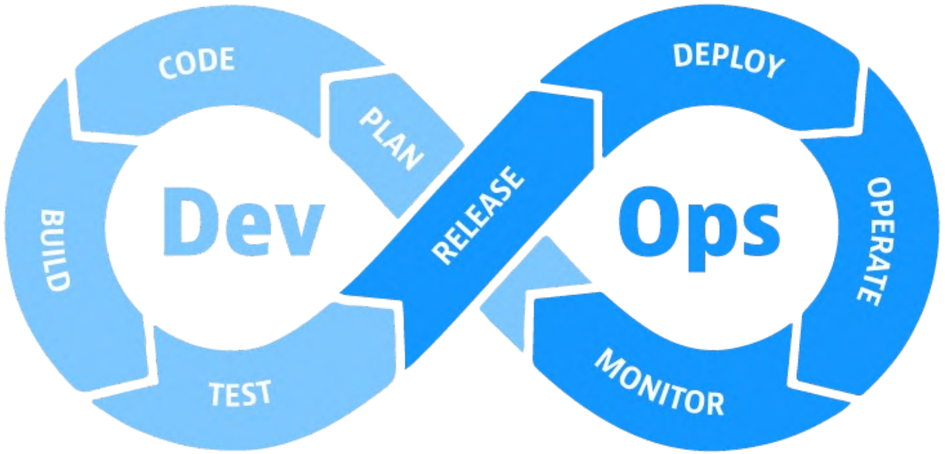
+

Operations Engineer

=

DevOps

DevOps



- Collaboration and communication
- Continuous integration
- Continuous delivery
- Infrastructure as Code (IaC)
- Automation
- Monitoring and feedback loops
- Agile and lean practices

DRIM

Security Engineer

- Security infrastructure design
- Vulnerability assessment and penetration testing
- Incident response and forensics
- Security monitoring and alerting
- Security policies and procedures
- Security awareness and training
- Security audits and compliance
- Research and staying updated

DevSecOps

Software Engineer

+

Operations Engineer

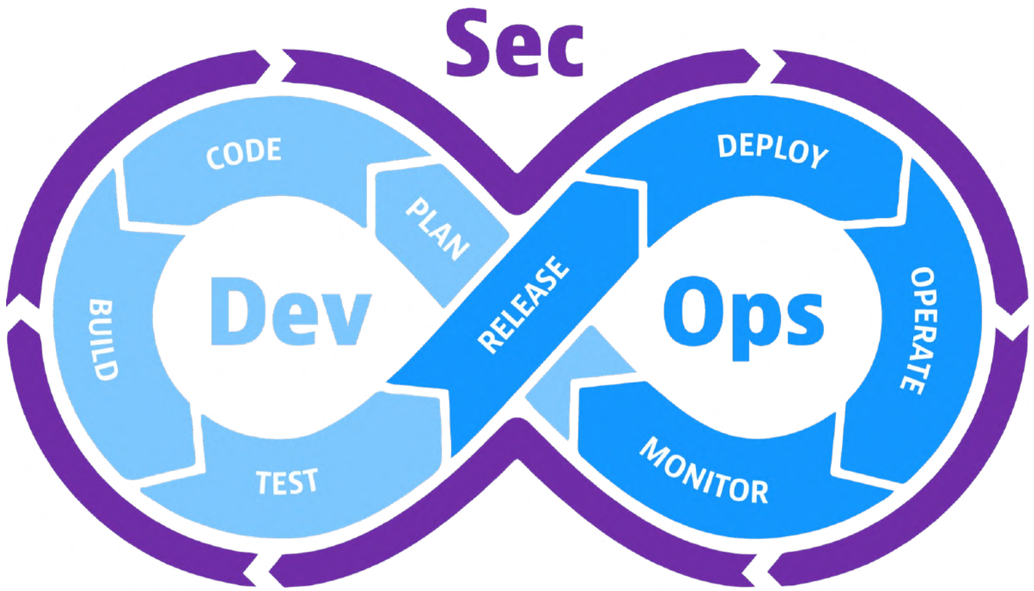
+

Security Engineer

=

DevSecOps

DevSecOps



- Collaboration and shared responsibility
- Automation
- Shift-left security
- Continuous Security
- Security as Code
- Continuous monitoring and feedback
- Culture of security

DRIM

Properties of Software Systems

Properties of Software Systems

- Reliability
- Scalability
- Maintainability

Reliability

- The application performs the function that the user expected
- It can tolerate the user making mistakes or using the software in unexpected ways
- Its performance is good enough for the required use case, under the expected load and data volume
- The system prevents any unauthorized access and abuse

Scalability

System scalability refers to the ability of a system to handle increasing amounts of work or data without experiencing a significant decrease in performance or stability. In simpler terms, it means that a system can efficiently and effectively accommodate growth in terms of users, data, or workload without encountering major issues or bottlenecks.

Scalability

- Shared-nothing architecture
- Modularity
- Redundancy
- Distributed computing
- Caching and optimization
- Elasticity

Maintainability

- Operability

  Make it easy for operations teams to keep the system running smoothly
- Simplicity

  Make it easy for new engineers to understand the system, by removing as much complexity as possible from the system
- Evolvability

  Make it easy for engineers to make changes to the system in the future, adaptingit for unanticipated use cases as requirements change

DRIM

Software Engineering Principles

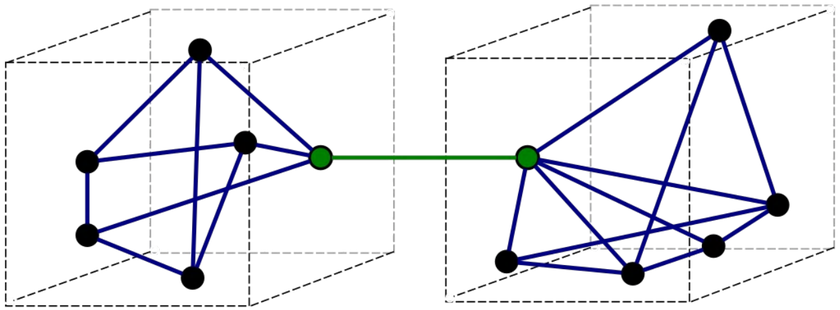
Software Engineering Principles

- Simplicity
- Loose Coupling + High Cohesion
- Single Responsibility
- Don't Repeat Yourself
- Coding to Contract
- Open–Closed Principle
- Inversion of Control
- Refactoring

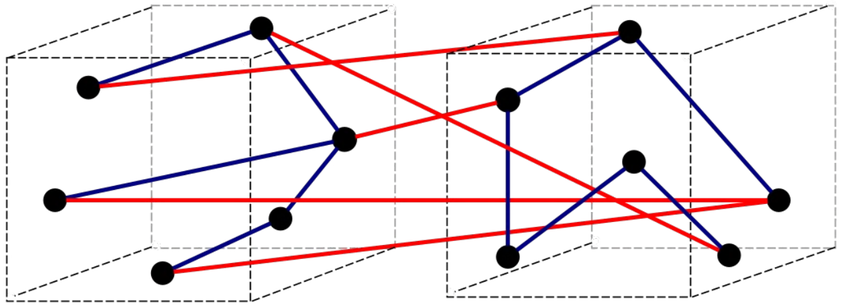
Simplicity

- YAGNI – You Aren't Gonna Need It
- KISS – Keep It Simple, Stupid!
- Make everything as simple as possible, but not simpler
- Simple vs. Easy; Complex vs. Hard – https://bit.ly/44SvQ7C
- Accidental Complexity vs. Inherent Complexity
- Hide Complexity and Build Abstractions
- Prevent Analysis Paralysis
- Prefer Composition over Inheritance

Loose Coupling + High Cohesion



a) Good (loose coupling, high cohesion)



b) Bad (high coupling, low cohesion)

- Minimal Dependencies
- Abstraction
- Modularity
- Encapsulation
- Composition
- Single Responsibility
- UNIX pipes

Single Responsibility

- Single Reason to Change
- Cohesion
- Separation of Concerns
- Reduced Dependencies
- Readability and Maintainability
- Testability
- Reusability

Don't Repeat Yourself

- Single Copy of Information
- Single Source of Truth
- Reduced Code Duplication
- Abstraction and Reusability
- Code Maintenance
- Readability and Understandability
- Testability

Coding to Contract

- Interfaces
- Modularity
- Encapsulation
- Interoperability
- Testability
- Maintainability

Open–Closed Principle

- Open for Extension but Closed for Modification
- Abstractions and Interfaces
- Modularity
- Maintainability

Inversion of Control

- Move Control from Component to Component's Owner
- Loose Coupling
- Extensibility
- Testability
- Dependency Injection

Inversion of Control

```
public class ReportController
{
    private readonly ReportGenerator _generator;
    private readonly ReportSender _sender;

    public ReportController()
    {
        _generator = new ReportGenerator();
        _sender = new ReportSender();
    }

    public void ProcessReport()
    {
        var report = _generator.Generate();
        _sender.Send(report);
    }
}

2 usages
public class ReportGenerator
{
    1 usage
    public Report Generate()
    {
        //...
    }
}

2 usages
public class ReportSender
{
    1 usage
    public void Send(Report report)
    {
        //...
    }
}
```

```
public class ReportController
{
    private readonly IReportGenerator _generator;
    private readonly IReportSender _sender;

    public ReportController(IReportGenerator generator, IReportSender sender)
    {
        _generator = generator;
        _sender = sender;
    }

    public void ProcessReport()
    {
        var report = _generator.Generate();
        _sender.Send(report);
    }
}

2 usages
public interface IReportGenerator
{
    1 usage
    Report Generate();
}

2 usages
public interface IReportSender
{
    1 usage
    void Send(Report report);
}
```

Refactoring

- Restructuring Existing Code without Changing its Behavior
- One Step at a Time
- Make it Work then Refactor
- Cleaner Design
- Cleaner Code

DRIM

Thank you! Questions?

